

Assignment11 – Natural Language

Given: July 1 Due: July 6

Problem 11.1 (Language Models)

Consider a language L over the alphabet with characters a, b, c, d , and consider a corpus D for it consisting of the five words

$abccc \quad abbd a \quad a a b c \quad b c d b \quad a a b a c$

You want to build a 3-gram model for L .

1. How many 3-grams are there over L ?

Solution: $4^3 = 64$

2. Now assume you store the 3-gram model as a Bayesian network with variables w_1 , w_2 , and w_3 (representing the 3-gram $w_1 w_2 w_3$), how many entries does the conditional probability table for w_3 have?

Solution: $444 = 64$ or $443 = 48$ if we utilize redundancy. Either answer was accepted.

3. Give the numerical value of $P(w_3 = a \mid w_{1:2} = aa)$ in the resulting model.

Solution: $1/3$

4. Give the numerical value of $\text{tf}(a, cdad)$.

Solution: $\frac{1}{4}$

5. Give the numerical value of $\text{idf}(a, D)$.

Solution: $\log_{10} 5/4$

6. Now you want to use the tfidf vector embedding. What are the objects that are embedded, and how are the vectors computed using tf and idf ?

Solution: We embed documents/words W from L . The vector for W is $\text{tf}(x, W) \cdot \text{idf}(x, D)$ for $x \in \{a, b, c, d\}$.

7. Now assume you are interested in all documents that contain the subword bc . A query system returns all 5 documents.
Give the numerical values of

1. precision: \$3/5\$
2. recall: \$1\$

Solution: filled in above

Problem 11.2 (Information Retrieval)

1. Explain (in about 2 sentences) the connection between **tfidf** and **cosine similarity** to relate texts.

Solution: **tfidf** is a way to represent a text as a vector, relative to a **corpus**, by measuring the frequency of terms in some way. **Cosine similarity** uses the **angle** between two such vectors as a measure of how similar the texts represented by the vectors are.

2. Explain (in about 2 sentences) the difference between **information retrieval** and **information extraction**.

Solution: **Information retrieval** identifies documents that are likely to contain information that answers the query. **Information extraction** obtains **structured representations** of parts of the content of a document, which can then be used to answer queries precisely.

3. Explain (in about 3 sentences) what kind of function is learned in the continuous **bag of words** algorithm, as used e.g. in **Word2Vec**, and how the training proceeds.

Solution: It learns a **word embedding**, i.e., a function from **words** to vectors. Going through a **corpus**, each **word** is processed together with the context, i.e., the n **words** before and after. The neural network is trained to predict the **word** from its context.

Problem 11.3 (Grammars)

Consider the following **probabilistic grammar**:

<i>S</i>	→	<i>NP VP</i> [1]
<i>NP</i>	→	<i>Article Noun</i> [0.6] <i>Name</i> [0.4]
<i>VP</i>	→	<i>Verb</i> [0.5] <i>TransVerb NP</i> [0.5]
<i>Article</i>	→	the[0.7] a[0.3]
<i>Noun</i>	→	stench[0.2] breeze[0.3] wumpus[0.5]
<i>Name</i>	→	John[0.3] Mary[0.7]
<i>Verb</i>	→	smells[1]
<i>TransVerb</i>	→	sees[0.6] shoots[0.4]

1. Using this **grammar** as an example, explain the difference between **structural rule** and **lexicon**.

Solution: Both are **productions** of the **grammar**. **Structural rule** define the **language** in general (*S* to *Article*, above), the **lexicon** defines the specific **words** used in a context (*Noun* to *TransVerb* above).

2. Give the **probability** of the **sentence**

John sees the wumpus

(You have to give the **expression** with concrete **values** plugged in, but you do not have to **compute** the result.)

Solution: $0.40.30.50.60.60.70.5 = 0.00756$

3. Now assume we do not know the **probabilities** of the **production**, and our **corpus** is

John sees the wumpus. The wumpus smells. John shoots the wumpus.

Give the **probability** that we can learn for $NP \rightarrow Name$ from this **corpus**.

Solution: We count all **subtrees** of type *NP* (*N*) and among those the ones of type *Name* (*n*), then we learn the **probability** $n \div N$. That yields $\frac{2}{5}$.
